

# PW EARNERS DSA in C++

## DPP — 2 (SOLUTIONS)

Topic: Conditionals (If-Else, Logical Operators, Ternary, Switch)

*For Q1 to Q10: Predict the exact output of each program block. If there is a compilation error or undefined behavior, state the reason clearly.*

**Q1.**

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 5, b = 10;
5     if(a == b) {
6         cout << "Equal matches " << a << endl;
7     } else {
8         cout << "Not equal " << a << endl;
9     }
10    return 0;
11 }
```

**Output:**

Not equal 5

**Explanation:** The '==' operator checks for equality. Since 5 is not equal to 10, the condition is false, and the program executes the **else** block.

.....

**Q2.**

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int a = 5, b = 10;
5     if(a = b) {
6         cout << "Equal matches " << a << endl;
7     } else {
8         cout << "Not equal " << a << endl;
9     }
10    return 0;
11 }
```

**Output:**

Equal matches 10

**Explanation:** Note the single '=' inside the **if** statement. This assigns the value of *b* (10) to *a*. The statement then evaluates **if(10)**, which is considered **true** in C++ (any non-zero value is true). Therefore, it prints the updated value of *a*.

.....

## Q3.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int num = -7;
5     if (num >= 0) {
6         cout << "Inside Positive block" << endl;
7     } else {
8         cout << "Inside Negative block" << endl;
9     }
10    return 0;
11 }

```

**Output:**

Inside Negative block

**Explanation:**  $-7$  is not greater than or equal to 0. The condition fails, passing execution to the else block.

## Q4. (Concept: Short-circuit evaluation)

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 0, y = 5;
5     if(x != 0 && (y = y + 10) > 10) {
6         cout << "Inside if branch" << endl;
7     }
8     cout << "x = " << x << ", y = " << y << endl;
9     return 0;
10 }

```

**Output:**

x = 0, y = 5

**Explanation:** This question demonstrates the **Short-Circuiting** behavior of the logical AND (&&) operator. For an && condition to be true, *both* sides must be true.

- The program first evaluates the left side:  $x \neq 0$ . Since  $x$  is 0, this is **false**.
- Because the first part is false, C++ knows the entire && condition can never be true.
- Therefore, it completely **skips** evaluating the right side  $(y = y + 10) > 10$  to save time.
- As a result,  $y$  is never incremented, and remains 5.

## Q5.

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int p = 1, q = 20;
5     if(p == 1 || (q = q + 5) > 25) {
6         cout << "First branch hit" << endl;
7     }
8     cout << "p = " << p << ", q = " << q << endl;
9     return 0;

```

```
10 }
```

**Output:**

First branch hit

$p = 1, q = 20$

**Explanation:** Similar to Q4, this demonstrates **Short-Circuiting**, but for the logical OR (`||`) operator. For an `||` condition to be true, only *one* side needs to be true.

- The program evaluates the left side: `p == 1`. Since  $p$  is indeed 1, this is **true**.
- Because the left side is true, the entire `||` condition is guaranteed to be true, regardless of what the right side is.
- C++ skips the right side (`q = q + 5`)  $> 25$ . Thus,  $q$  is never updated and remains 20.

.....

**Q6.**

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int marks = 45;
5     bool pass = (marks >= 40) ? true : false;
6     int bonus = pass ? 5 : 0;
7     cout << "Pass status: " << pass << ", Bonus: " << bonus << endl;
8     return 0;
9 }
```

**Output:**

Pass status: 1, Bonus: 5

**Explanation:** This code uses the **Ternary Operator** (`condition ? true_value : false_value`).

- First, `(marks >= 40)` evaluates to true ( $45 \geq 40$ ). So, `pass` becomes **true**. When printed in C++, **true** displays as 1.
- Second, `bonus = pass ? 5 : 0;`. Because `pass` is true, it selects the value before the colon (5).
- So, the final output prints 1 for `pass`, and 5 for the `bonus`.

.....

**Q7.**

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 12, y = 8;
5     int max_val = (x > y) ? x : y;
6     cout << "Maximum value is " << max_val << endl;
7     return 0;
8 }
```

**Output:**

Maximum value is 12

**Explanation:** The ternary condition checks  $12 > 8$ , which is true. Therefore, it assigns  $x$  (12) to `max_val`.

.....

**Q8. (Concept: Switch fall-through effect)**

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int code = 2;
5     switch(code) {
6         case 1: cout << "Alpha" << endl;
7         case 2: cout << "Beta" << endl;
8         case 3: cout << "Gamma" << endl;
9             break;
10        default: cout << "Omega" << endl;
11    }
12    return 0;
13 }

```

**Output:**

Beta

Gamma

**Explanation:** This highlights the **Fall-Through Property** of switch statements.

- The code jumps directly to case 2 and prints "Beta".
- However, there is **no break statement** at the end of case 2. Without a **break**, C++ does not exit the switch; it just keeps executing the code in the cases directly below it.
- Execution "falls through" to case 3, printing "Gamma".
- It finally encounters the **break** statement inside case 3, stopping the switch execution before it reaches default.

**Q9.**

```

1 #include<iostream>
2 using namespace std;
3 int main(){
4     int val = 65;
5     switch(val) {
6         case 'A': cout << "Character match A" << endl; break;
7         case 65: cout << "Integer match 65" << endl; break;
8         default: cout << "No match found" << endl;
9     }
10    return 0;
11 }

```

**Output:**

Compilation Error

**Reason & Explanation:** In C++, characters are stored as their underlying ASCII integer values. The ASCII value for 'A' is 65. Therefore, the compiler sees case 'A': and case 65: as having the exact same value. Since a switch statement cannot contain duplicate cases, it throws an error.**Q10. (Crazy Question)**

```

1 #include<iostream>
2 using namespace std;
3 int main(){

```

```

4   int a = 0, b = 0;
5   if (a++ && ++b) {
6       cout << "Condition standard check true" << endl;
7   }
8   cout << "a = " << a << ", b = " << b << endl;
9   return 0;
10 }
```

**Output:**

a = 1, b = 0

**Explanation:** This combines Post-Increment operators and Short-Circuiting. It can be tricky, so let's break it down:

- The condition is `a++ && ++b`. It evaluates the left side first: `a++`.
- `a++` is a **post-increment**. This means it uses the *current* value of `a` for the evaluation, and *then* increments it.
- The current value of `a` is 0 (which represents false). The left side evaluates to false, and then `a` immediately becomes 1.
- Because the left side of the `&&` operator evaluated to false, **short-circuiting** kicks in! The right side (`++b`) is entirely ignored.
- The result? The `if` block does not run, `a` has been incremented to 1, and `b` was never touched, remaining 0.

.....

*For Q11 to Q20: Write clean, modular, and compilable C++ programs. Ensure proper use of data types, validation where necessary, and formatted user prompts.*

**Q11.** Take two integers as input from the user. Determine and print whether their multiplication product is positive, negative, or zero without calculating the absolute numeric valuation value if possible.

*Sample* — Input: -4 5 ⇒ Output: Product is Negative

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int n1, n2;
5      cin >> n1 >> n2;
6      // If either is zero, the product is zero.
7      if (n1 == 0 || n2 == 0) {
8          cout << "Product is Zero" << endl;
9      }
10     // If both share the same sign (+,+ or -,-), product is positive
11     } else if ((n1 > 0 && n2 > 0) || (n1 < 0 && n2 < 0)) {
12         cout << "Product is Positive" << endl;
13     }
14     // Otherwise, signs are opposite, so product is negative.
15     } else {
16         cout << "Product is Negative" << endl;
17     }
18     return 0;
19 }
```

**Explanation:** Instead of performing the potentially large multiplication (which could cause

an overflow), we use logical checks on the signs of the numbers. Negative times negative equals positive!

**Q12.** Take a positive integer input from the user and check if it is divisible by both 3 and 5. Do **not** use the logical AND (&&) operator inside your conditional test statement.

*Sample* — Input: 30  $\Rightarrow$  Output: Divisible by both 3 and 5

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6     // 15 is the Least Common Multiple (LCM) of 3 and 5.
7     if (n % 15 == 0) {
8         cout << "Divisible by both 3 and 5" << endl;
9     } else {
10        cout << "Not divisible by both 3 and 5" << endl;
11    }
12    return 0;
13 }
```

**Explanation:** In math, if a number is divisible by two coprime numbers (like 3 and 5), it is perfectly divisible by their product (15). Using the modulo operator (%) checks for a remainder of 0.

**Q13.** Given the number of sides  $n$  of a simple polygon and the sum of all its interior angles  $S$ , write a program to check if the user has provided valid dimensional specifications. Use the structural formula:  $S = (n - 2) \times 180^\circ$ .

*Sample* — Input: Sides = 3, Sum = 180  $\Rightarrow$  Output: Valid Polygon

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n, S;
5     cin >> n >> S;
6     // A polygon must have at least 3 sides.
7     if (n > 2 && S == (n - 2) * 180) {
8         cout << "Valid Polygon" << endl;
9     } else {
10        cout << "Invalid Polygon Specifications" << endl;
11    }
12    return 0;
13 }
```

**Explanation:** We first check if  $n > 2$  because a 1-sided or 2-sided polygon does not exist. Then we simply match their input sum  $S$  against the mathematical formula.

**Q14.** Take two integers as input. If exactly one of them is odd, print "One is odd". If both are odd, print "Both are odd". If neither of them is odd, print "None is odd".

*Sample* — Input: 4 7  $\Rightarrow$  Output: One is odd

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a, b;
5      cin >> a >> b;
6      // Each condition resolves to 1 (true) or 0 (false).
7      // Adding them mathematically counts how many are true!
8      int oddCount = (a % 2 != 0) + (b % 2 != 0);
9
10     if (oddCount == 1) {
11         cout << "One is odd" << endl;
12     } else if (oddCount == 2) {
13         cout << "Both are odd" << endl;
14     } else {
15         cout << "None is odd" << endl;
16     }
17     return 0;
18 }

```

**Explanation:** This uses a clever trick. The expression `(a % 2 != 0)` returns boolean `true` (1) or `false` (0). By adding the two booleans together, we easily get a total count (0, 1, or 2) of odd numbers.

.....

**Q15.** Write a menu-driven application using a C++ `switch-case` structure that accepts an integer number representing a month (1 to 12). Print the corresponding month's standard calendar name.

*Sample* — Input: 2  $\Rightarrow$  Output: February

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int month;
5      cin >> month;
6      switch(month) {
7          case 1:  cout << "January" << endl; break;
8          case 2:  cout << "February" << endl; break;
9          case 3:  cout << "March" << endl; break;
10         case 4:  cout << "April" << endl; break;
11         case 5:  cout << "May" << endl; break;
12         case 6:  cout << "June" << endl; break;
13         case 7:  cout << "July" << endl; break;
14         case 8:  cout << "August" << endl; break;
15         case 9:  cout << "September" << endl; break;
16         case 10: cout << "October" << endl; break;
17         case 11: cout << "November" << endl; break;
18         case 12: cout << "December" << endl; break;
19         default: cout << "Invalid month scope" << endl;
20     }
21     return 0;
22 }

```

**Explanation:** A `switch` statement is perfect here as we have discrete integer values. We make

sure to use `break` to prevent fall-through!

### Q16. (Concept Booster: Upper/Lower Boundary constraints)

Take a character input from the keyboard. Write a conditional structure to evaluate whether the provided letter is an uppercase English alphabet (A-Z), a lowercase alphabet (a-z), a numerical digit (0-9), or a special symbolic character.

Sample — Input: g ⇒ Output: Lowercase Alphabet

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     char ch;
5     cin >> ch;
6     if (ch >= 'A' && ch <= 'Z') {
7         cout << "Uppercase Alphabet" << endl;
8     } else if (ch >= 'a' && ch <= 'z') {
9         cout << "Lowercase Alphabet" << endl;
10    } else if (ch >= '0' && ch <= '9') {
11        cout << "Numerical Digit" << endl;
12    } else {
13        cout << "Special Symbolic Character" << endl;
14    }
15    return 0;
16 }
```

**Explanation:** In C++, you can compare characters directly using '`>=`' and '`<=`'. The compiler checks their ASCII decimal values behind the scenes.

### Q17. (Concept Booster: Leap Year Calculation Rule)

Write a program that takes a positive integer representing a year (e.g., 2024) and checks whether it is a leap year or not.

Sample — Input: 1900 ⇒ Output: Not a Leap Year

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     int year;
5     cin >> year;
6     if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
7         cout << "Leap Year" << endl;
8     } else {
9         cout << "Not a Leap Year" << endl;
10    }
11    return 0;
12 }
```

**Explanation:** Leap year logic is slightly trickier than just "divisible by 4".

- **Rule 1:** If a year is a multiple of 4, it is generally a leap year (`year % 4 == 0`).
- **Rule 2:** However, "century years" (like 1700, 1800, 1900) are **not** leap years, which is why we add `&& year % 100 != 0`.
- **Rule 3:** But, there is an exception to the exception! If a century year is perfectly divisible



by 400 (like 1600 or 2000), it *is* a leap year after all (`|| year % 400 == 0`).

### Q18. (Concept Booster: Coordinate System Analysis)

Given the three coordinate points  $A(x_1, y_1)$ ,  $B(x_2, y_2)$ , and  $C(x_3, y_3)$  as floating-point inputs, write a program to determine if all three points sit sequentially on the same straight line (collinear points).

Sample — Input: 0 0 2 2 4 4  $\Rightarrow$  Output: Points are Collinear

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     double x1, y1, x2, y2, x3, y3;
5     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
6
7     // Using cross-multiplication for slopes to avoid division by
8     // zero.
9     if ((y2 - y1) * (x3 - x2) == (y3 - y2) * (x2 - x1)) {
10         cout << "Points are Collinear" << endl;
11     } else {
12         cout << "Points are Not Collinear" << endl;
13     }
14     return 0;
15 }
```

**Explanation:** For three points to be strictly on the same straight line, the *slope* between point A and point B must equal the slope between point B and point C.

- The standard slope formula is:  $\frac{y_2 - y_1}{x_2 - x_1} = \frac{y_3 - y_2}{x_3 - x_2}$ .
- However, in programming, using division is dangerous because if the points form a vertical line, the denominators ( $x_2 - x_1$  or  $x_3 - x_2$ ) will be exactly 0, crashing your program!
- **Pro Tip:** We cross-multiply the equation to check it via multiplication instead:  $(y_2 - y_1) \times (x_3 - x_2) = (y_3 - y_2) \times (x_2 - x_1)$ . This is perfectly safe from zero-division errors.

### Q19. (Concept Booster: Triangle Categorization Code)

Take three numerical values denoting the side dimensions ( $a, b, c$ ) of a shape. First, check if valid, further categorize as an **Equilateral**, **Isosceles**, or **Scalene** triangle.

Sample — Input: 5 5 8  $\Rightarrow$  Output: Isosceles Triangle

```

1 #include <iostream>
2 using namespace std;
3 int main() {
4     double a, b, c;
5     cin >> a >> b >> c;
6
7     // Triangle Inequality Theorem validation
8     if (a + b > c && b + c > a && c + a > b) {
9         if (a == b && b == c) {
10             cout << "Equilateral Triangle" << endl;
11         } else if (a == b || b == c || c == a) {
12             cout << "Isosceles Triangle" << endl;
13         } else {
14             cout << "Scalene Triangle" << endl;
15         }
16     }
17 }
```

```

15     }
16     } else {
17         cout << "Not a Valid Triangle" << endl;
18     }
19     return 0;
20 }

```

**Explanation:** First, we must validate the inputs using the Triangle Inequality Theorem: the sum of the lengths of any two sides must be strictly greater than the third side. If true, we check equality pairs for the specific type.

### Q20. (Concept Booster: Electric Bill Multi-Tier Grid Pricing)

Compute dynamic electricity charges with an additional structural safety surcharge of 15% appended to the total amount generated.

*Sample* — Input: 150 units  $\Rightarrow$  Output: Total Bill = \$316.25

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      double units;
5      cin >> units;
6      double baseBill = 0;
7
8      // Slab processing
9      if (units <= 100) {
10         baseBill = units * 1.50;
11     } else if (units <= 300) {
12         baseBill = (100 * 1.50) + (units - 100) * 2.50;
13     } else {
14         baseBill = (100 * 1.50) + (200 * 2.50) + (units - 300) *
15             4.00;
16     }
17
18     // 15% surcharge calculation (baseBill + (baseBill * 0.15))
19     double totalBill = baseBill * 1.15;
20     cout << "Total Bill = $" << totalBill << endl;
21     return 0;
22 }

```

**Explanation:** This code uses a "slab rate" structure (like taxes). You max out one lower tier before getting charged the rate for the next tier. Finally, multiplying the result by 1.15 is a mathematical shortcut to calculate the base bill plus an extra 15% surcharge.

All the best! Keep Coding ♡